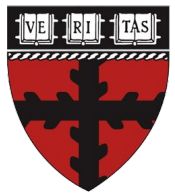
Lecture 20: Deployment: Pulumi

AC215/E115

Pavlos Protopapas
SEAS/Harvard



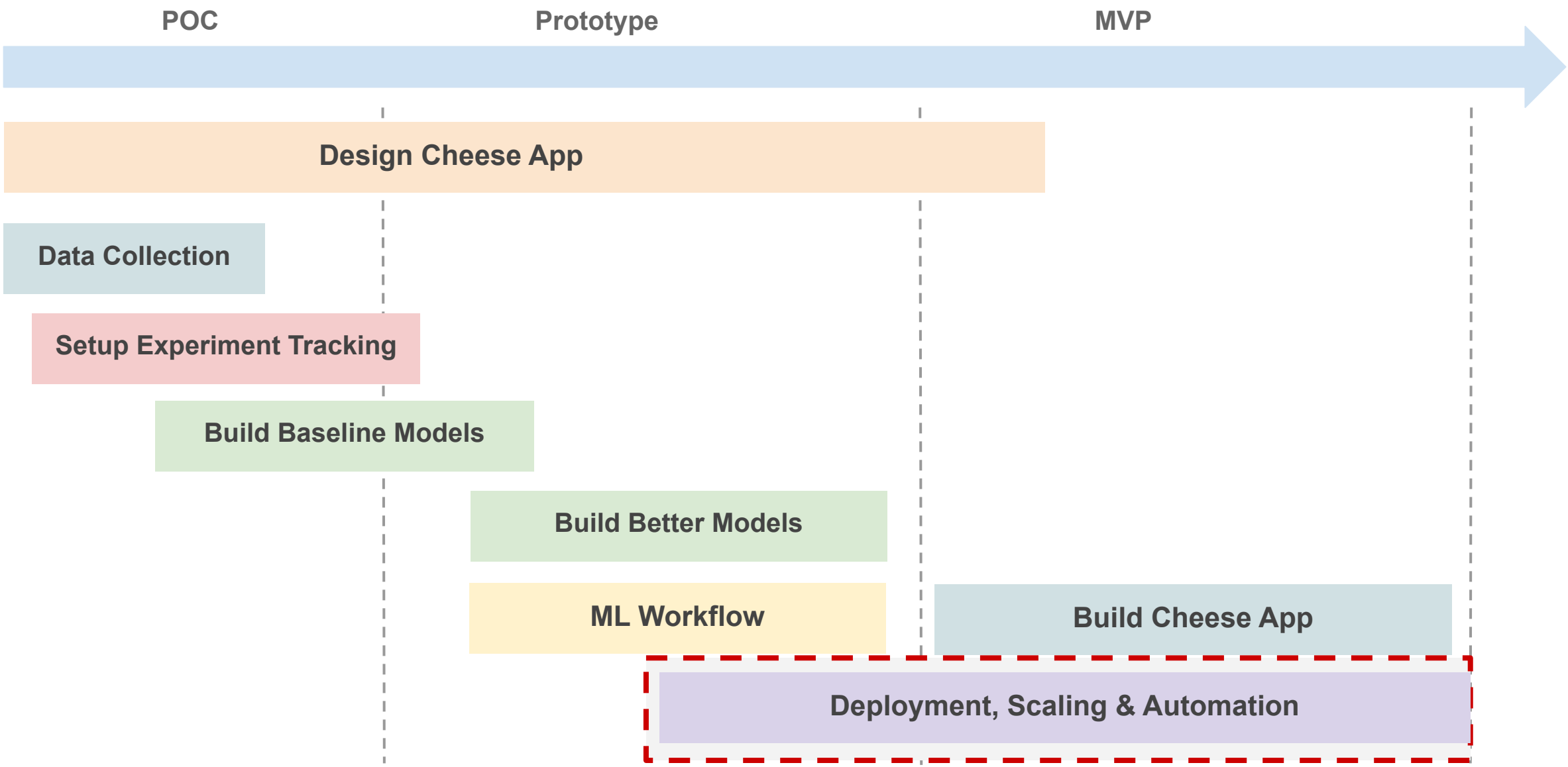
Outline

1. Recap
2. Deployment

Outline

1. **Recap**
2. Deployment

Recap: Cheese App Status



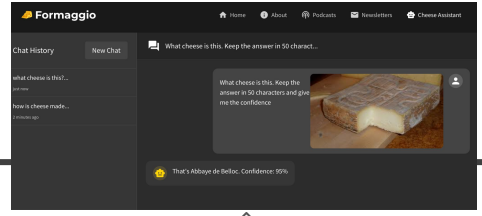
Recap: APIs & Frontend App

- Everything we built is on our **local computer**
- We need to **deploy** this to a server so our users can access

Outline

1. Recap
2. **Deployment**

Deployment: Goal



HTTP 80

GCP

Compute Instance (Virtual Machine)

HTTP 9000

NGINX Container

HTTP 3000

API Service Container

Cheese Front-End App Container

HTTP 8000

Vector DB Container

Local Computer

IDE/ CLI

Containers

HTTPS 443

NFS

GCS Bucket

Models

GCE Persistent Volume

Local Models, Experiments

Tutorial: Deployment Steps (Manual)

Push to Docker Hub: Build and push Docker images for API service and Frontend to Docker Hub repository

Setup VM Environment: Create VM instance on GCP and install Docker, create required folders and set permissions

Deploy Containers: Run Docker containers for API service and Frontend, creating a dedicated network for communication

Configure Web Server: Setup Nginx as reverse proxy, configure routing for API and Frontend services

Cheese App - Deployment to GCP (Manual)



Deployment Automation

In our manual deployment there were various steps to keep track of.

We want to automate this!



Pulumi

- Is a tool for infrastructure automation
- Think of infrastructure as code
- Pulumi programs use real programming languages (Python, TypeScript, Go, C#, Java)
- Pulumi code consists of instructions for tasks like
  - Server & Cluster creation/deletion
  - Software installation & setup
  - Networking setup
- Everything is code, so you can check it into GitHub and share

Pulumi: History

- Pulumi was founded in 2017 by **Joe Duffy, Eric Rudder, and Luke Hoban**.
- It manages infrastructure using general-purpose programming languages like Python, TypeScript, Go, C#, and Java.
- Programs programs run from a single machine (your workstation or CI/CD runner) and communicate with cloud providers through their APIs.
- Infrastructure is defined using real code with functions, classes, and loops, not YAML or DSL (Domain Specific Language).

Pulumi: Principles

Real Programming Languages

Declarative Infrastructure

Multi-Cloud Support



Component-Based Architecture

State Management & Previews

Test-Driven Infrastructure

Pulumi: Concepts

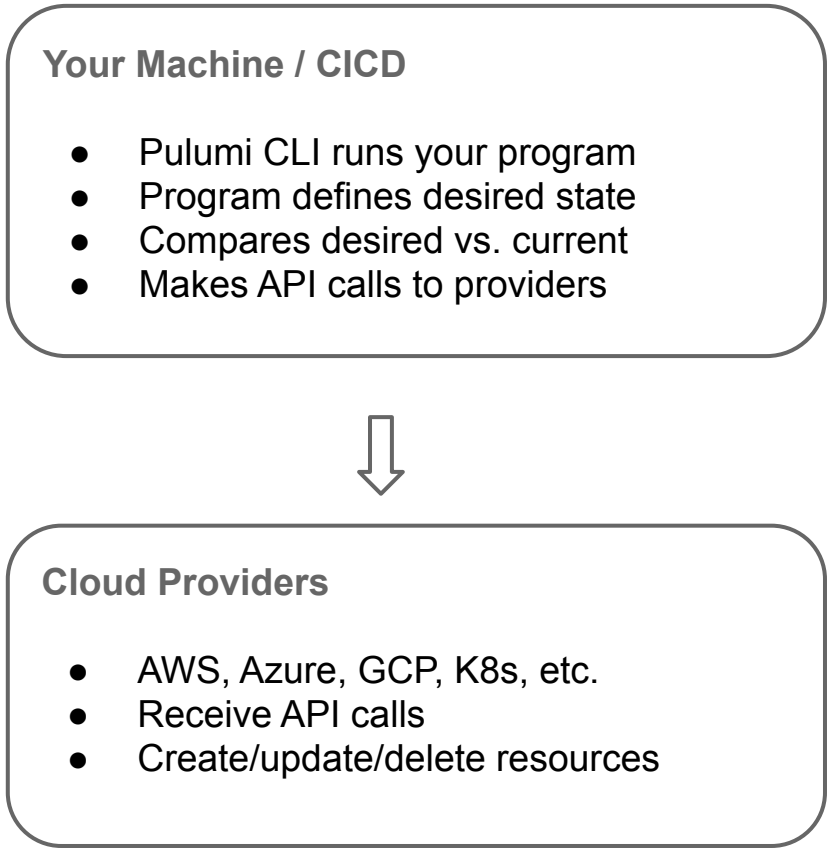
- Pulumi CLI Commands
- Programs
- Stacks
- Resources
- State
- Providers

Pulumi: Concepts

Pulumi programs describe your desired infrastructure using **real programming languages** like Python, TypeScript, Go, C#, or Java.

Pulumi **executes** your program locally and communicates with cloud providers through their APIs to create and manage resources.

How Pulumi Works



PULUMI MANAGES INFRASTRUCTURE THROUGH CLOUD APIs

Pulumi: Commands

Common Pulumi Commands

- **pulumi new <template>** - *Create a new Pulumi project from a template*
- **pulumi up** - *Preview and deploy infrastructure changes*
- **pulumi preview** - *Preview changes without deploying*
- **pulumi destroy** - *Tear down all resources in the stack*
- **pulumi stack** - *Manage stacks (environments)*
- **pulumi config** - *Manage configuration values*
- **pulumi refresh** - *Refresh the state to match actual infrastructure*

Pulumi: Running Programs

Deploying Infrastructure with Pulumi

pulumi up [options]

- Executes your Pulumi program to determine desired infrastructure state
- Shows a preview of changes (resources to create, update, or delete)
- Prompts for confirmation before applying changes
- --yes: Skip confirmation and automatically approve changes
- --stack <name>: Specify which stack (environment) to deploy to
- --refresh: Refresh state before previewing changes

Pulumi: Programs

- Pulumi programs define **infrastructure as code** using real programming languages
- You write code using **familiar constructs** like functions, classes, loops, and conditionals
- Programs are **declarative** - you describe what you want, Pulumi figures out how to create it

```python
# Example: Create a GCS bucket in Google Cloud
import pulumi
import pulumi_gcp as gcp

# Create a GCS bucket
bucket = gcp.storage.Bucket("my-bucket",
    location="US",
    website=gcp.storage.BucketWebsiteArgs(
        main_page_suffix="index.html",
    )
)
```

Pulumi: Stacks

- A **stack** is an isolated instance of your infrastructure (like dev, staging, production)
- Each stack has its own **state** and **configuration**
- Stacks let you deploy the same program to multiple environments
- You can have different configuration values for each stack
- Common stack naming: **<project>-<environment>** (e.g., cheese-app-dev, cheese-app-prod)

Deployment Steps (Pulumi / Automation)

1. Setup local container to connect to GCP
2. Build and push docker images to GAR
3. Create Compute Instance (VM) in GCP
4. Provision the server (Installed required softwares)
5. Setup Docker containers in VM Instance
6. Setup a web server to expose our app to the outside world

Setup local container /GCP

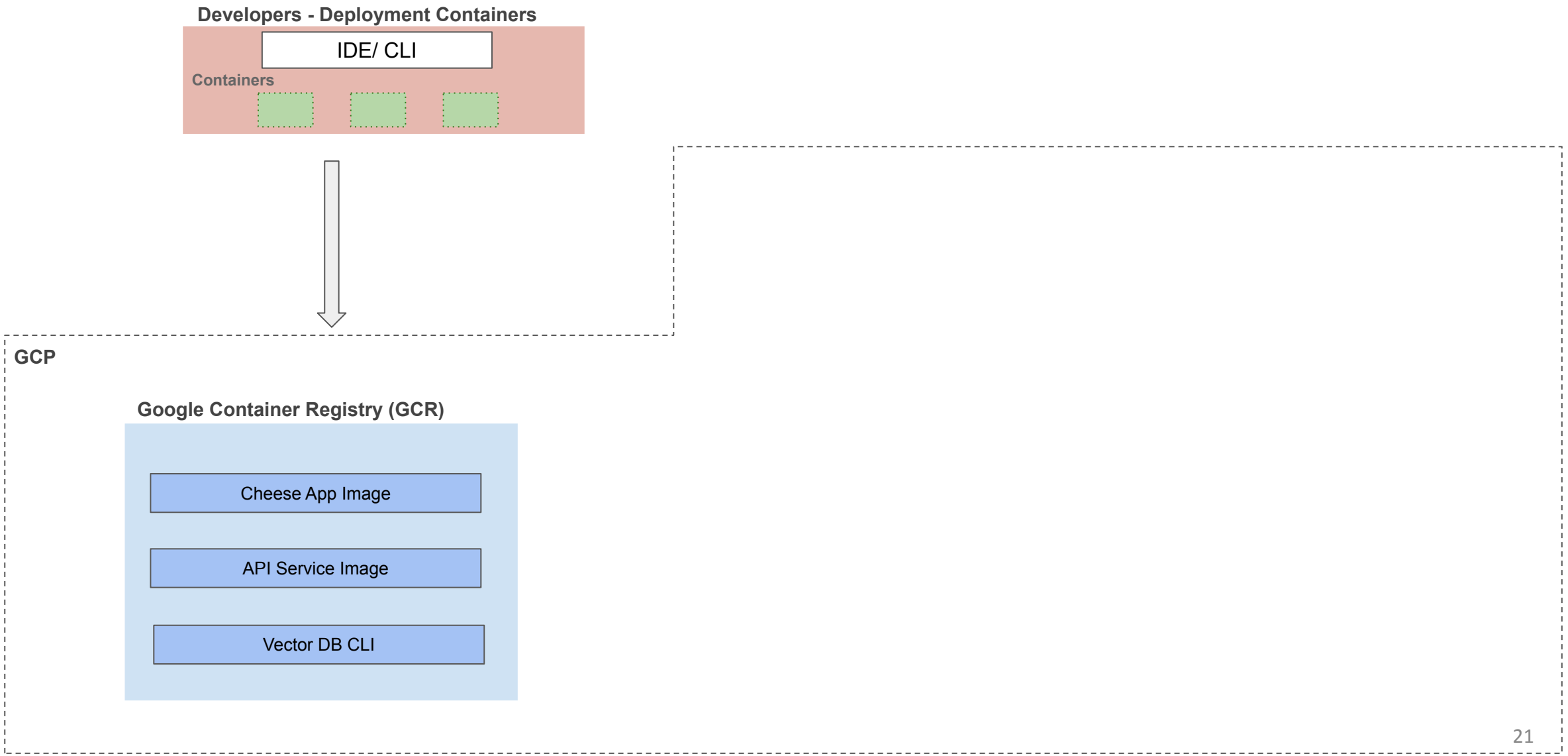
Setup required GCP

- Enable APIs
- Create service accounts
  - **deployment** (To deploy everything to GCP)
  - **gcp-service** (To read containers from GAR in VM)

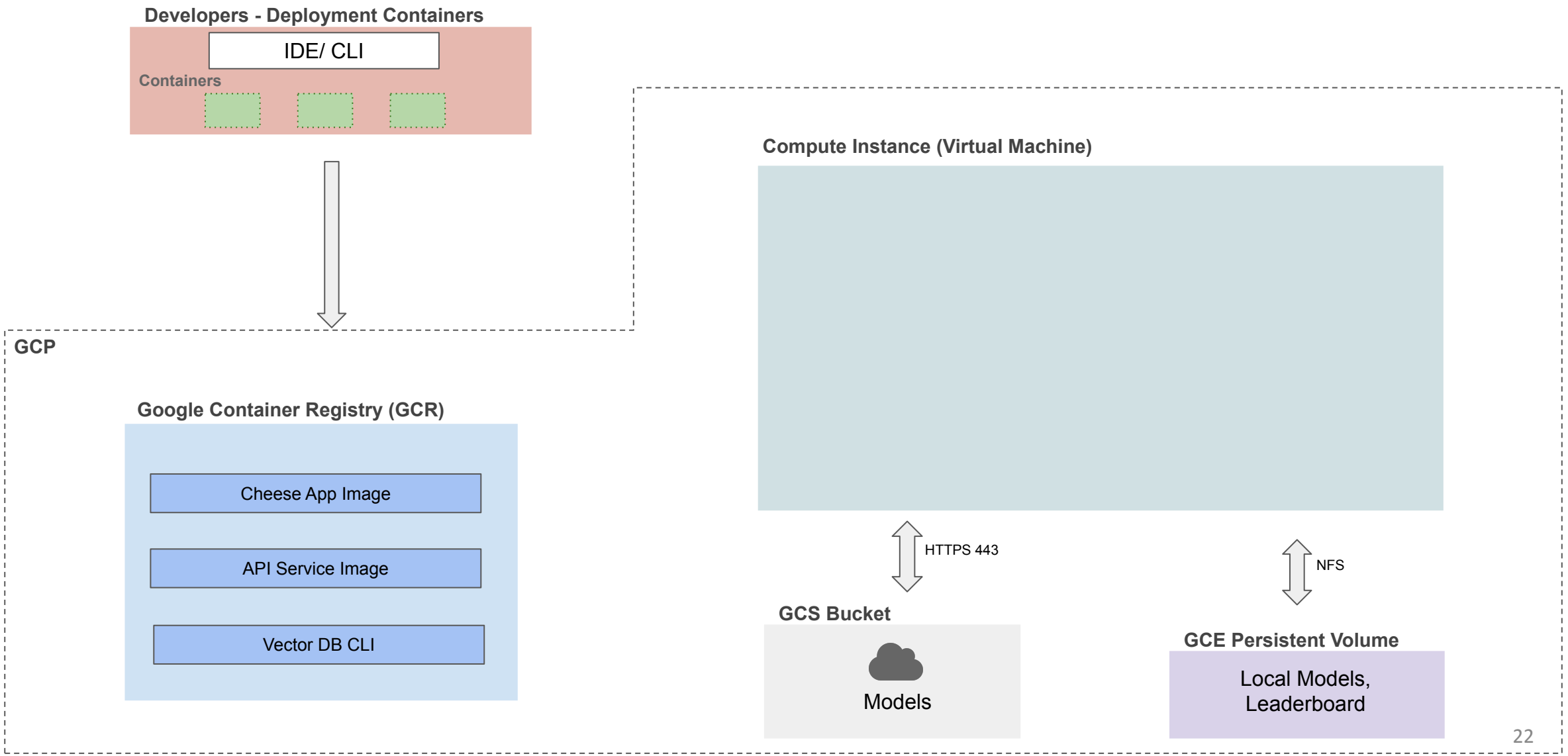
Setup local deployment container

- Add secret keys
- Set GCP project we want to connect to

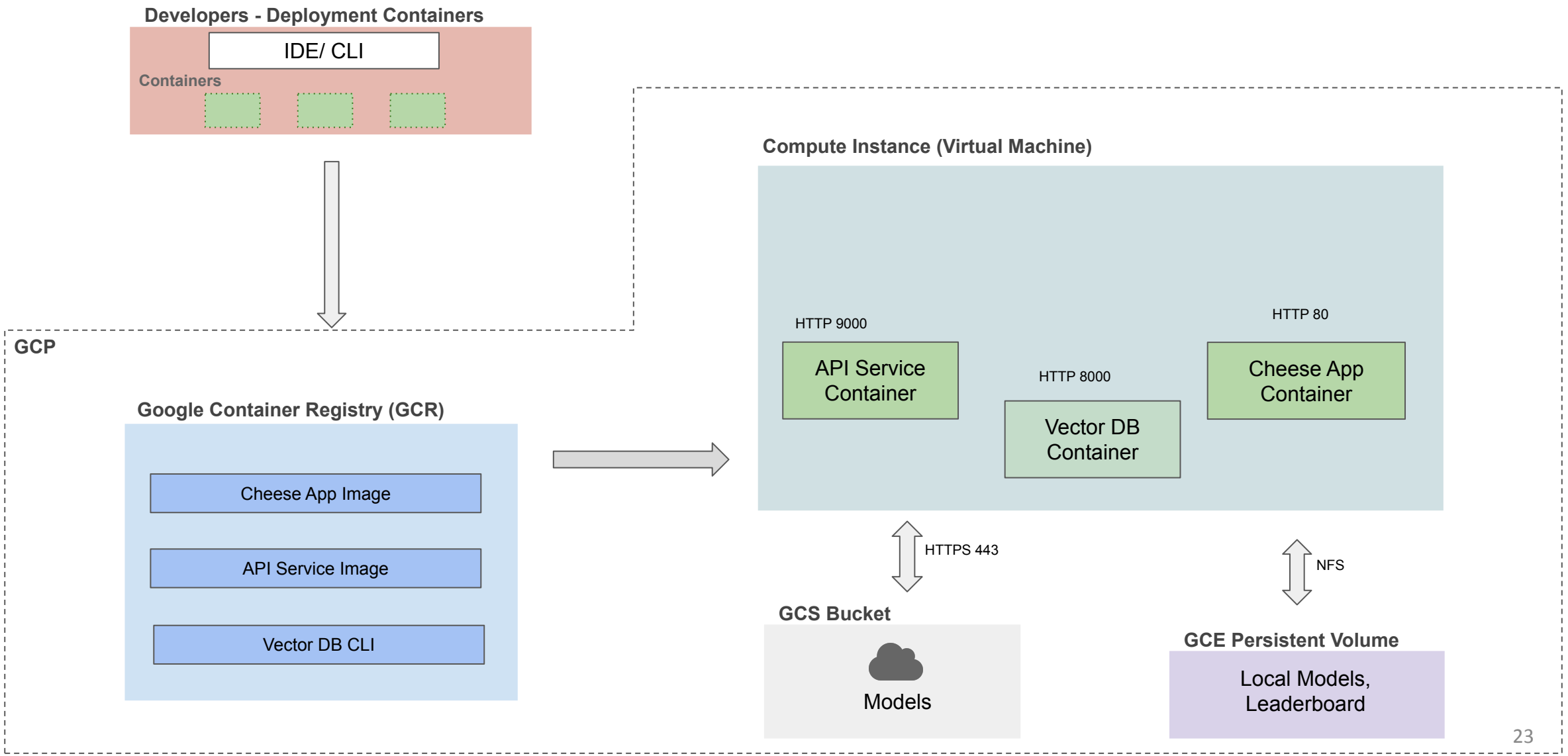
Build & Push Docker Images to GCR



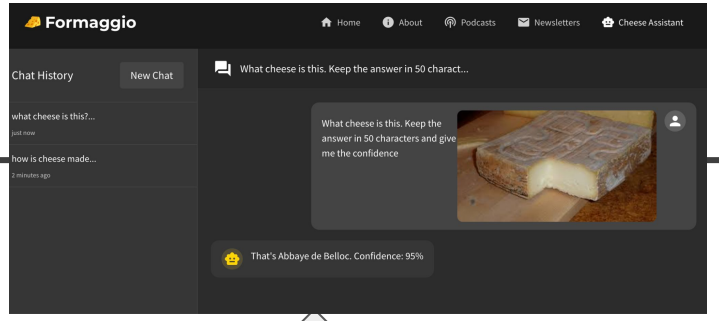
Create Compute Instance (VM)



Setup Docker Containers in VM



Setup Web Server to expose App



HTTP 80

Developers - Deployment Containers

IDE/ CLI

Containers

GCP

Compute Instance (Virtual Machine)

HTTP 9000

NGINX Container

HTTP 3000

API Service Container

Cheese App Container

HTTP 8000

Vector DB Container

Google Container Registry (GCR)

Cheese App Image

API Service Image

Vector DB CLI

HTTPS 443

NFS

GCS Bucket

Models

GCE Persistent Volume

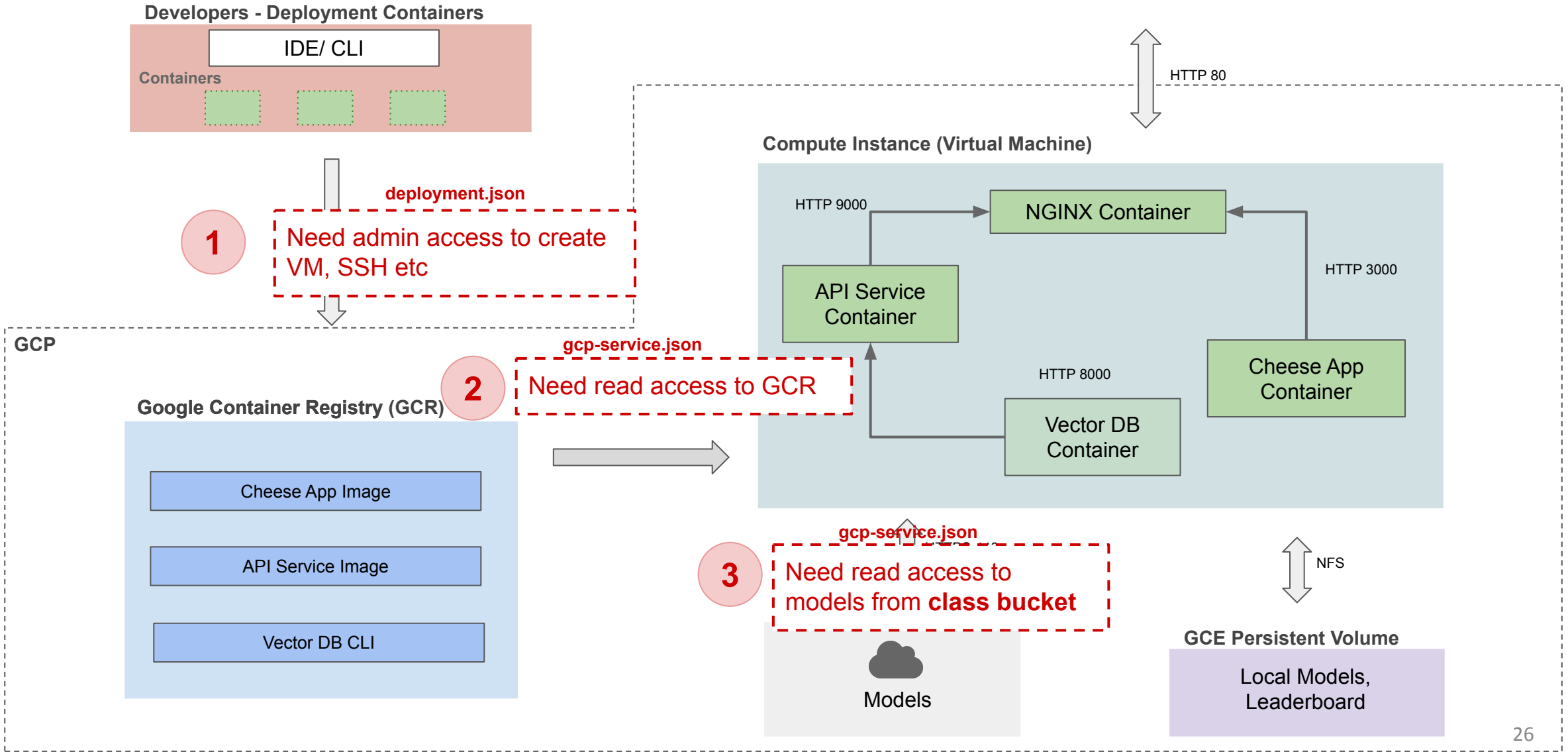
Local Models, Leaderboard

Why did we need 2 service accounts?

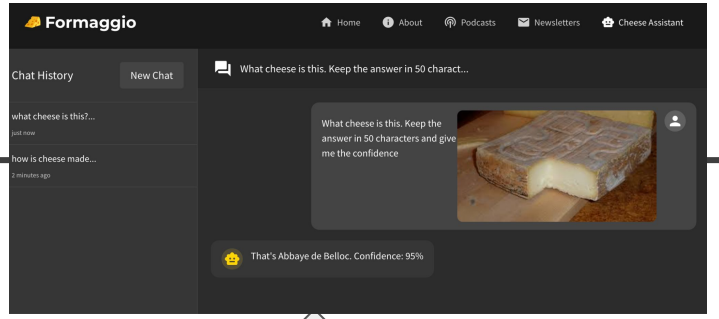
Why do we need the following service accounts?

- **deployment**
  - Has admin access to **your** group GCP project
- **gcp-service**
  - Has read access to **your** group GCP projects GCR
  - Has access to Vertex AI to perform inference

Why did we need 2 service accounts?



Test the App



HTTP 80

Developers - Deployment Containers

IDE/ CLI

Containers

GCP

Compute Instance (Virtual Machine)

HTTP 9000

NGINX Container

HTTP 3000

API Service Container

HTTP 8000

Cheese App Container

Vector DB Container

Google Container Registry (GCR)

Cheese App Image

API Service Image

Vector DB CLI

HTTPS 443

NFS

GCS Bucket

Models

GCE Persistent Volume

Local Models, Leaderboard

Tutorial: Deployment to GCP (Pulumi)

Cheese App - Deployment to GCP (Pulumi)

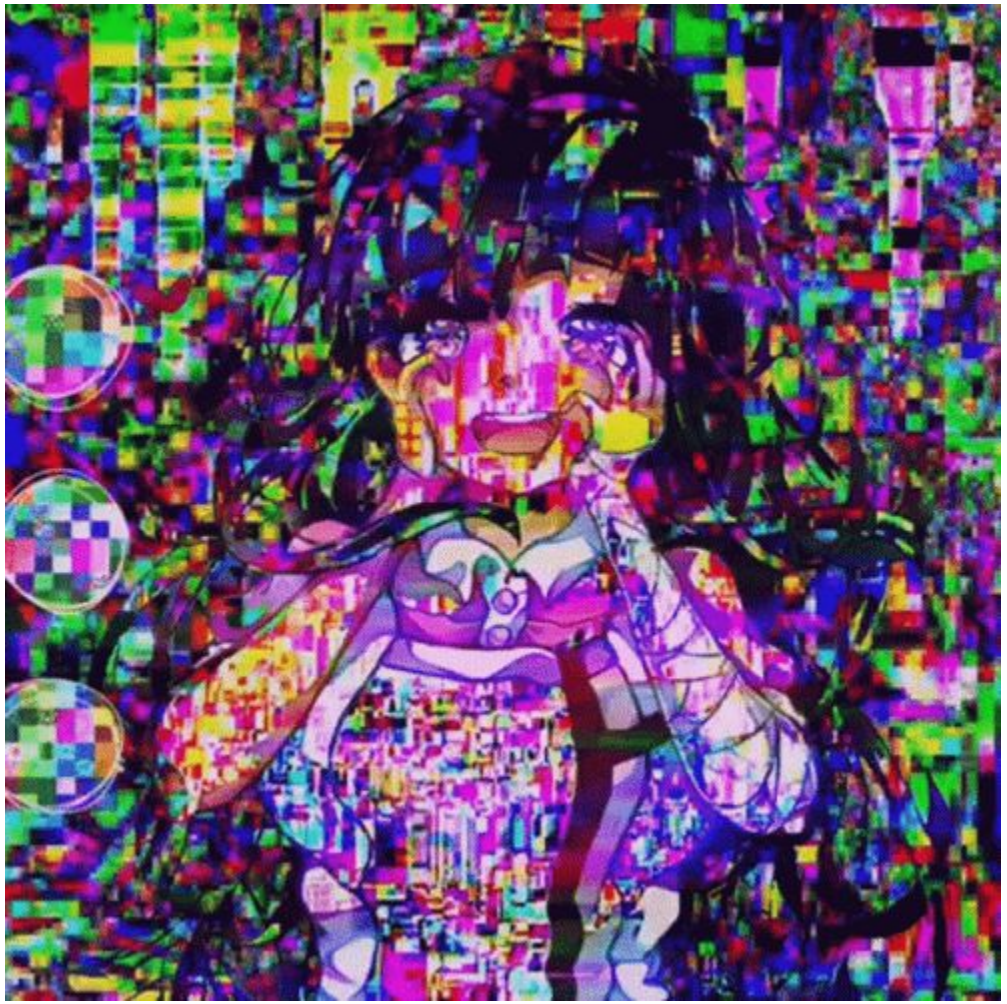
https://github.com/dlops-io/cheese-app-v3?tab=readme-ov-file#deployment-to-gcp-single-vm-instance

Build & Push Images: Build and push Docker containers to Google Container Registry (GCR)

Create VM Instance: Deploy a Compute Engine VM

Provision Instance: Install required dependencies and setup environment

Deploy Application: Configure and launch Docker containers (API, Frontend, Nginx)



DON’T FORGET TO DELETE


THANK YOU!!!